

CLOUD-08: Multi-Cloud Observability Patterns

Series: CLOUD | Notebook: 8 of 8 | Created: March 2026 | Last Updated: 03/12/2026

Overview

This notebook covers strategies for building unified observability across multiple cloud providers with Dynatrace. You will learn how to create cross-cloud dashboards, establish consistent naming conventions, build unified alerting, compare costs across providers, and implement governance patterns for multi-cloud environments.

Table of Contents

- 1. [Multi-Cloud Challenges](#)
- 2. [Unified Entity Model](#)
- 3. [Cross-Cloud Comparison Queries](#)
- 4. [Consistent Naming Conventions](#)
- 5. [Unified Health Dashboards](#)
- 6. [Multi-Cloud Alerting](#)
- 7. [Cost Comparison Patterns](#)
- 8. [Governance Across Providers](#)
- 9. [Summary](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	<code>metrics.read</code> , <code>entities.read</code> , <code>logs.read</code>
Cloud Integrations	At least two cloud providers configured (AWS, Azure, or GCP)
Prior Knowledge	CLOUD-01 through CLOUD-06

1. Multi-Cloud Challenges

Organizations running workloads across multiple cloud providers face these observability challenges:

Challenge	Description
Terminology mismatch	AWS "instances", Azure "VMs", GCP "instances" are all compute but named differently
Metric inconsistency	CPU metrics have different units, granularity, and collection methods
Entity fragmentation	Same application may span multiple providers with no inherent linking
Alert fatigue	Separate alerting per provider leads to duplicate or conflicting alerts
Cost opacity	Each provider has different billing models, making comparison difficult
Skill silos	Teams specialize in one provider and lack visibility into others

How Dynatrace Helps

Dynatrace provides a **unified entity model** that normalizes cloud resources across providers:

Dynatrace Concept	AWS	Azure	GCP
<code>dt.entity.host</code>	EC2 Instance	Virtual Machine	Compute Instance
<code>dt.entity.service</code>	ECS Service / Lambda	App Service	Cloud Run / GKE Service
<code>dt.entity.kubernetes_cluster</code>	EKS	AKS	GKE
Tags	AWS Tags	Azure Tags	GCP Labels
<code>dt.host.cpu.usage</code>	CloudWatch CPU	Azure Monitor CPU	Cloud Monitoring CPU

2. Unified Entity Model

Dynatrace normalizes cloud entities into a unified model. This enables cross-cloud queries using the same DQL regardless of provider.

Cross-Cloud Entity Types

Entity Type	Covers
<code>dt.entity.host</code>	EC2, Azure VM, GCE instances (with OneAgent)
<code>dt.entity.service</code>	Any auto-detected service across all providers
<code>dt.entity.process_group</code>	Processes across all hosts/containers
<code>dt.entity.kubernetes_cluster</code>	EKS, AKS, GKE clusters
<code>dt.entity.kubernetes_node</code>	Worker nodes across all K8s providers

Cloud-Specific Entity Types

Some entities are provider-specific and require separate queries:

AWS	Azure	GCP
dt.entity.ec2_instance	dt.entity.azure_vm	(uses dt.entity.host)
dt.entity.aws_lambda_function	dt.entity.azure_web_app	dt.entity.cloud_application
dt.entity.relational_database_service	dt.entity.azure_sql_database	(custom device)

3. Cross-Cloud Comparison Queries

Compute Resource Count by Provider

```
// Compare compute resource counts across cloud providers
fetch dt.entity.ec2_instance
| summarize provider = "AWS", resource_type = "EC2 Instance", resource_count = count()
| append [
  fetch dt.entity.azure_vm
  | summarize provider = "Azure", resource_type = "Virtual Machine", resource_count = count()
]
| append [
  fetch dt.entity.aws_lambda_function
  | summarize provider = "AWS", resource_type = "Lambda Function", resource_count = count()
]
| append [
  fetch dt.entity.azure_web_app
  | summarize provider = "Azure", resource_type = "Web App", resource_count = count()
]
| sort provider asc, resource_count desc
```

Unified Host CPU Usage (All Providers)

```
// Top 10 hosts by CPU usage across all cloud providers
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Kubernetes Cluster Comparison

```
// List all Kubernetes clusters (EKS, AKS, GKE)
fetch dt.entity.kubernetes_cluster
| fieldsKeep id, entity.name, tags
| sort entity.name asc
```

Cross-Cloud Problem Analysis

```
// Davis problems from the last 7 days across all infrastructure
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| summarize problem_count = count(), avg_duration_hours = avg(toLong(resolved_problem_duration) / 3600000000000.0), by:{event.category}
| sort problem_count desc
```

Unified Log Analysis

```
// Error log count by source across all providers in the last 6 hours
fetch logs, from:-6h
| filter loglevel == "ERROR"
| summarize error_count = count(), by:{log.source}
| sort error_count desc
| limit 15
```

4. Consistent Naming Conventions

A naming standard is critical for multi-cloud environments. Apply consistent conventions across all providers.

Recommended Tag Schema

Tag Key	Purpose	Example Values
cloud-provider	Identify the source provider	aws , azure , gcp
environment	Deployment environment	prod , staging , dev
team	Owning team	platform , backend , data
application	Application name	checkout , catalog , auth
cost-center	Finance allocation	CC-1234 , engineering
region	Normalized region name	us-east , eu-west , ap-south

Region Normalization

Cloud providers use different region naming:

Normalized	AWS	Azure	GCP
us-east	us-east-1	eastus	us-east1
us-west	us-west-2	westus2	us-west1
eu-west	eu-west-1	westeurope	europe-west1
ap-south	ap-southeast-1	southeastasia	asia-southeast1

Use a normalized `region` tag in Dynatrace to enable cross-cloud region-based queries.

5. Unified Health Dashboards

Build dashboards that show health across all providers in a single view.

Dashboard Components

Component	Query Type	Purpose
Infrastructure health	<code>timeseries avg(dt.host.cpu.usage)</code>	CPU/memory across all hosts
Active problems	<code>fetch dt.davis.problems</code>	Open problems across all infrastructure
Service health	<code>fetch spans with error rate</code>	Service-level error rates
Log errors	<code>fetch logs with loglevel filter</code>	Error log trends
Kubernetes health	<code>timeseries dt.kubernetes.*</code>	Container metrics across K8s clusters

Active Problems Across All Infrastructure

```
// Active problems grouped by category
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| summarize active_count = countDistinct(event.id), by:{event.category}
| sort active_count desc
```

Service Error Rate (Unified)

```
// Service-level error rates from spans in the last hour
fetch spans, from:-1h
| filter span.kind == "server"
| summarize total = count(), errors = countIf(span.status_code == "ERROR"), by:{service.name}
| fieldsAdd error_pct = errors * 100.0 / total
| filter total > 10
| sort error_pct desc
| limit 15
```

6. Multi-Cloud Alerting

Alerting Strategy

Level	Scope	Alert Type	Example
Platform	All providers	Davis AI anomaly	Unexpected CPU spike on any host
Provider	Single cloud	Metric threshold	AWS Lambda error rate > 5%
Application	Cross-cloud	Custom metric	Checkout service p95 > 2s
Compliance	All providers	Configuration	Untagged resources detected

Best Practices

Practice	Description
Use Davis AI	Let Dynatrace auto-detect anomalies across all infrastructure
Normalize severity	Map provider-specific severities to a unified scale
Route by team, not provider	Alert the application team regardless of which cloud has the issue
Deduplicate	Davis automatically correlates related issues across providers
Define SLOs at the service level	SLOs should be cloud-agnostic

7. Cost Comparison Patterns

While Dynatrace is not a cost management tool, you can use monitoring data to inform cost decisions.

Resource Utilization Comparison

```
// Host utilization over the last 24 hours - identify underutilized hosts
timeseries avgCpu = avg(dt.host.cpu.usage), from:-24h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| filter avgCpuValue < 10
| sort avgCpuValue asc
| limit 20
```

Cost Optimization Indicators

Indicator	What It Means	Action
CPU < 10% sustained	Overprovisioned instance	Rightsize or terminate
Memory < 20% sustained	Overprovisioned memory	Rightsize instance type
Lambda avg duration near timeout	Function at risk	Optimize code or increase timeout
Zero traffic services	Unused resources	Decommission candidates
K8s pods with no requests/limits	Uncontrolled resource usage	Enforce resource quotas

```
// Kubernetes namespaces with highest resource consumption (cost proxies)
timeseries nsCpu = avg(dt.kubernetes.container.cpu_usage), from:-24h, by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 10
```

8. Governance Across Providers

Governance Framework

Governance Area	Dynatrace Implementation
Access control	Segments by cloud provider + environment; IAM policies per team
Tagging compliance	Monitor for untagged entities; alert on non-compliance
Cost allocation	Tag-based DDU attribution per team/application
Security posture	Runtime vulnerability analysis across all monitored hosts
Change tracking	Davis events for configuration changes across providers

Tagging Compliance Check

```
// Find hosts with no tags (potential compliance issue)
fetch dt.entity.host
| fieldsKeep id, entity.name, tags
| filter isNull(tags) or tags == ""
| sort entity.name asc
| limit 20
```

Vulnerability Overview

```
// Davis security problems in the last 7 days
fetch dt.davis.problems, from:-7d
| filter event.category == "SECURITY"
| summarize security_count = countDistinct(event.id), by:{event.status}
| sort security_count desc
```

Multi-Cloud Governance Best Practices

Practice	Description
Enforce consistent tagging	Use automation to reject untagged resources
Create cloud-agnostic SLOs	Define SLOs based on user experience, not infrastructure
Centralize alert routing	Single Dynatrace workflow routes alerts to the right team
Regular utilization reviews	Monthly review of underutilized resources across all providers
Unified runbooks	Runbooks should reference Dynatrace queries, not provider-specific tools

9. Summary

Key Takeaways

- Dynatrace's **unified entity model** normalizes cloud resources across AWS, Azure, and GCP
- **Cross-cloud queries** use the same DQL patterns via shared entity types like `dt.entity.host` and `dt.entity.service`
- **Consistent tagging** (`cloud-provider` , `environment` , `team` , `application`) is the foundation of multi-cloud governance
- **Alerting should be team-based**, not provider-based — route to the application owner regardless of which cloud has the issue
- Use **utilization data** to identify cost optimization opportunities across all providers

Series Complete

This notebook concludes the CLOUD series. For deeper dives into specific areas, explore:

- **K8S series** — Advanced Kubernetes monitoring patterns
- **OPLOGS series** — OpenPipeline log processing
- **IAM series** — Enterprise IAM administration
- **WFLOW series** — Workflows and alert notifications

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.